

INSTITUTO NACIONAL DE APRENDIZAJE

Núcleo Comercio y Servicios · Subsector Informática y Comunicación

Diseño de algoritmos

Sesión 16 · Lenguaje C: formato, compilación y errores

UA2 · Programación estructurada | Módulo CSTI12010



Persona facilitadora

Giovanni Antonio Coto Calderón

Ruta de la sesión



1. Inicio

Repaso y preguntas
diagnósticas

20 min



2. Desarrollo

Exposición · práctica
guiada · práctica
autónoma

175 min



3. Síntesis

Cacería de errores en
parejas (coevaluación)

20 min



4. Próxima sesión

Variables, tipos de datos,
constantes y entrada y
salida

5 min

Recordemos la sesión anterior



Repaso 1

¿Qué hace git commit?

¿Y git push?



Repaso 2

¿Por qué el archivo hola.exe no se sube a GitHub?

Recuerde el archivo .gitignore.

Antes de empezar, piense y responda



Pregunta 1

¿Qué diferencia hay entre PSeInt y gcc?

Piense en intérprete y compilador (sesión 1).



Pregunta 2

Si escribe Printf en lugar de printf, ¿funcionará?

¿Importan las mayúsculas en PSeInt? ¿Y en C?

Capacidad 2.1 y saberes de la sesión

Capacidad procedimental 2.1

Emplear los principales elementos semánticos y sintácticos del lenguaje de desarrollo en el paradigma de programación estructurada.



Del Ser: Autonomía

Valor: Honradez



Del Convivir: Comunicación asertiva

Valor: Respeto

Saberes



1. El lenguaje C



2. De PSeInt a C



3. Formato de un programa



4. Proceso de compilación



5. Tipos de errores



6. Leer mensajes de gcc

El lenguaje C



C es un lenguaje de programación de **alto nivel**, **compilado** y **estructurado**, creado por Dennis Ritchie en los Laboratorios Bell a inicios de los años 70 para escribir el sistema operativo UNIX.



Estructurado

Secuencia, decisión y repetición.



Compilado

gcc lo traduce a un ejecutable.



Portable

El mismo código compila en otros sistemas.



Base de otros

C++, Java, C# y PHP heredan su sintaxis.

De PSeInt a C: el mismo algoritmo

La lógica es la misma que en la UA1; cambian las **palabras** y las **reglas de escritura**. Ambos muestran: **Hola, INA / Bienvenidos a la UA2** (PSeInt) y **Hola, INA / Bienvenidos a la UA2** (gcc).

Saludo.psc

```
Algoritmo Saludo
    Escribir "Hola, INA";
    Escribir "Bienvenidos a la UA2";
FinAlgoritmo
```

saludo.c

```
#include <stdio.h>

int main(void) {
    printf("Hola, INA\n");
    printf("Bienvenidos a la UA2\n");
    return 0;
}
```

Anatomía de un programa en C

```
hola.c

/* hola.c
   Primer programa de la sesión 16 */
#include <stdio.h>           // entrada y salida

int main(void) {            // aquí empieza
    printf("Hola, INA!\n"); // muestra un mensaje
    return 0;               // termina sin errores
}
```



Comentarios

`/*` varias líneas `*/` y `//` una línea.



#include <stdio.h>

Trae printf (entrada y salida).



int main(void) { }

Función principal: aquí empieza.



return 0;

Informa al sistema: todo salió bien.

Reglas de formato de C



Distingue mayúsculas

`printf` ≠ `Printf`; `main` ≠ `Main`.



Sangría de 4 espacios

No es obligatoria, pero sí profesional.



Punto y coma

Cada instrucción termina en ;



Secuencias de escape

`\n` salto · `\t` tabulador · `\"` comillas · `\\` barra.



Llaves { }

Abren y cierran bloques; siempre en pareja.

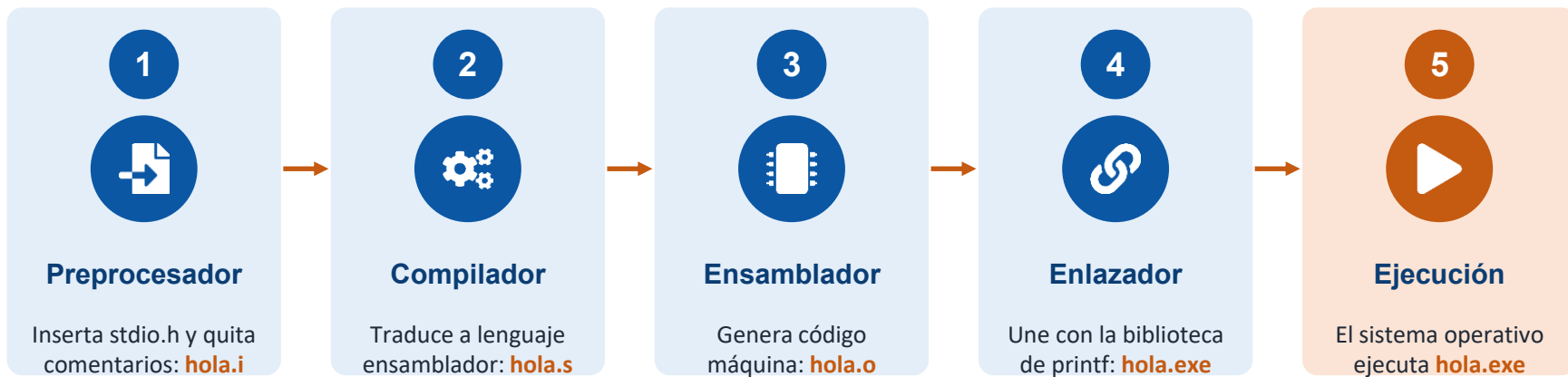


Un solo main

Todo programa tiene exactamente uno.

El proceso de compilación

Cuando se ejecuta `gcc hola.c -o hola`, ocurren cuatro etapas:



Relación con la sesión 1: el **ensamblador** y el **compilador** que estudiamos en teoría trabajan dentro de gcc.

gcc etapa por etapa

Comando	Se detiene después de...	Genera
<code>gcc -E hola.c -o hola.i</code>	Preprocesar	<code>hola.i</code> (texto)
<code>gcc -S hola.c</code>	Compilar	<code>hola.s</code> (texto)
<code>gcc -c hola.c</code>	Ensamblar	<code>hola.o</code> (binario)
<code>gcc hola.o -o hola</code>	Enlazar	<code>hola.exe</code>
<code>gcc -Wall -g hola.c -o hola</code>	Todo, con advertencias	<code>hola.exe</code>

En el curso se compila con **-Wall** (todas las advertencias): **Ctrl + Shift + B** en VS Code ejecuta `gcc -Wall -g`.

Tipos de errores

Tipo	Cuándo aparece	Ejemplo	¿Hay ejecutable?
Sintaxis	Al compilar	Falta un ; o una llave	No
Enlace	Al enlazar	Main o Printf mal escritos	No
Advertencia	Al compilar (-Wall)	Variable declarada y no usada	Sí
Ejecución	Al ejecutar	División entre cero	Sí, pero falla
Lógica	Al revisar resultados	Fórmula mal escrita	Sí, pero da mal

Cómo leer un mensaje de gcc

Formato: **archivo:línea:columna: tipo: mensaje**. Corrija siempre **el primer error** y vuelva a compilar.

```
falta.c

#include <stdio.h>

int main(void) {
    printf("Hola, INA!\n")
    return 0;
}
```

```
Mensaje real de gcc

falta.c: In function 'main':
falta.c:4:27: error: expected ';' before 'return'
   4 |         printf("Hola, INA!\n")
     |                                     ^
     |                                     ;
   5 |         return 0;
     |         ~~~~~
```

Mayúsculas que no enlazan

Si se escribe **Main** o **Printf**, el compilador no encuentra la función real y el enlazador falla con un mensaje que contiene **undefined reference**.



Nombres mal escritos

● ● ● No genera ejecutable

```
int Main(void) {  
    Printf("Hola\n");  
    return 0;  
}  
  
// ... undefined reference to ...  
// collect2: error: ld returned 1  
// exit status
```



Nombres correctos

● ● ● Compila y enlaza

```
int main(void) {  
    printf("Hola\n");  
    return 0;  
}  
  
// Hola
```

Advertencias: compila, pero algo sobra o falta

Con **-Wall**, gcc avisa de posibles problemas. El ejecutable **sí** se genera, pero en el curso se corrigen **todas** las advertencias.

```
aviso.c

#include <stdio.h>

int main(void) {
    int edad = 20;
    printf("Hola, INA!\n");
    return 0;
}
```

```
Advertencia real de gcc

aviso.c: In function 'main':
aviso.c:4:9: warning: unused variable 'edad' [-Wunused-variable]
    4 |     int edad = 20;
      |         ^~~~
```

El ciclo de trabajo en C



1. Editar

Escribir el código en VS Code.



2. Guardar

Ctrl + S (sin guardar, gcc compila la versión vieja).



3. Compilar

Ctrl + Shift + B o gcc -Wall.



4. Leer el primer mensaje

Línea, columna y tipo.



5. Ejecutar y probar

./programa.exe con casos de prueba.



6. Commit

Cuando funciona: git add, commit y push.

Paso a paso: del código fuente al ejecutable

A

Entorno listo

B

Programa con formato

C

Etapas de gcc

D

Cacería de errores

E

De PSeInt a C

Siga cada paso junto con la persona facilitadora y compruebe el resultado antes de avanzar.

Ahora usted: demuestre lo aprendido



Ejercicio 1 · Letrero con escapes

`\n, \t, \" y \\` en printf.



Ejercicio 2 · Corrija el programa

Cinco errores con ayuda de gcc.



Ejercicio 3 · De PSeInt a C

Traducir un algoritmo de salida.



Ejercicio 4 · Etapas de gcc

Archivos `.i`, `.s`, `.o` y `.exe`.

Documento: S16_Practica_autonoma_Estudiantes.docx · Entrega: carpeta UA2/S16/ del portafolio.

¿Qué aprendimos hoy?

1

¿Cuáles son las cuatro etapas de gcc?

2

¿Qué significan los números en falta.c:4:27?

3

¿Qué diferencia hay entre un error y una advertencia?

4

¿Qué hace `\t` dentro de `printf`?



Cacería de errores en parejas

Cada persona introduce 2 errores en el `letrero.c` de su pareja.

La pareja compila, lee los mensajes y corrige.

- ¿Encontró cada error por la línea y la columna?
- ¿Corrigió empezando por el primer mensaje?

Luego intercambian los roles.

Sesión 17: Variables, tipos de datos, constantes y entrada y salida



Tipos de datos

int, float, double, char



Constantes

#define y const



printf y scanf

Mostrar y leer datos

Para pensar: ¿qué tipo de C cree que corresponde a Entero, Real y Caracter de PSeInt?